

Gilded Rose

Legacy 코드 QA 프롬프팅

VSCode Java & C++ 개발자 | Cursor AI 단계별 QA 프롬프트 완전 가이드

1 요구사항 분석

2 코드 품질 분석

3 테스트 계획

4 테스트 케이스

5 테스트 실행·결함

6 Golden Master

7 리팩토링 지원

8 결함 관리·보고

9 QA 종합 검토

Gilded Rose — 실습 개요 & 브랜치 설정

실습 준비

실습 개요



프로젝트

기존 Gilded Rose_xx Repository 사용 (클론 없이 폴더 복사)



브랜치 전략

기존 PR은 merge 없이 유지 → main에서 prompting 브랜치 신규 생성



실습 목적

Martin Fowler Refactoring 기반 Cursor AI 단계별 QA 프롬프트 학습



Java 환경

Java 21 + JUnit 5 + Maven | VSCode Extension Pack for Java



C++ 환경

C++17 + Google Test + CMake | VSCode C/C++ + CMake Tools



VSCode 터미널 브랜치 설정 (Git Bash)

bash - 브랜치 생성 & 초기 커밋

```
# 1. 프로젝트 폴더 복사 후 VSCode에서 열기
#   (기존 Gilded Rose 폴더 → 새 경로로 복사)
code . # VSCode에서 프로젝트 열기

# 2. Terminal > New Terminal > Git Bash 선택

# 3. main 브랜치에서 prompting 브랜치 생성
git checkout main
git checkout -b prompting

# 4. 초기 커밋 & GitHub Push
git add .
git commit -m "chore: Cursor AI QA 프롬프팅 브랜치 초기화"
git push --set-upstream origin prompting
# → 첫 push 시 GitHub Token 인증 필요
#   Token 탭 선택 → PAT 입력 → Sign in
```



GildedRose 아이템 규칙 요약 (리뷰 전 필수 숙지)

일반 아이템

매일 quality -1 | 기한 지나면 -2 |
quality 최솟값 0

Aged Brie

오래될수록 quality ↑ | 기한 후 +2 |
quality 최댓값 50

Backstage Pass

sellIn<11→+2, <6→+3 | 기한 후
quality=0

Sulfuras

quality 항상 80 고정 | sellIn 변화 없음

Conjured ★

매일 quality -2 | 기한 후 -4 (일반의
2배 감소) ← 신규 기능

즉시 적용 가능한 핵심 프롬프트 3종 & .cursorrules 설정

빠른 시작

⚡ Cursor AI Chat(Ctrl+L) — 즉시 적용 가능한 프롬프트 (P-C-T-F 구조 적용)

① 문제점 식별 & 개선 제안

Java

[P] 당신은 레거시 코드 리팩토링 전문 시니어 Java 개발자입니다.
[C] @GildedRose.java @Item.java - Java 21, Maven, JUnit 5
[T] updateQuality() 메서드의 주요 문제점 3가지를 식별하고
각 문제별 리팩토링 개선 방안을 제시해줘.
[F] Markdown 표: 문제점 | 위반 원칙 | 개선 방안 | 우선순위

C++17

[C] @GildedRose.h @GildedRose.cpp - C++17, CMake, Google Test
[T] updateQuality() 함수의 문제점 3가지와 C++17 스타일 개선 방안.
std::variant 또는 strategy 패턴 적용 가능성도 검토해줘.

② 아이템 타입별 테스트 케이스 5개

Java

[C] @GildedRoseTest.java @GildedRoseRequirements.txt
[T] 각 아이템 타입별 최소 5개의 JUnit 5 테스트 케이스를 작성해줘:
일반/AgedBrie/BackstagePass/Sulfuras/Conjured
[F] @DisplayName("설명") + Given-When-Then 구조
경계값(0, 50, sellIn=0, sellIn=-1) 반드시 포함

C++17

[C] @GildedRoseTest.cpp - Google Test
[T] 각 아이템 타입별 최소 5개의 TEST_F 케이스 작성.
EXPECT_EQ / ASSERT_EQ 로 quality.sellIn 경계값 검증.

③ 리팩토링 안전장치 테스트 설계

Java

[C] @GildedRose.java @TexttestTestFixture.java
[T] 리팩토링 전 기능 회귀 방지를 위한 Golden Master 안전장치 테스트를 설계해줘:
1. TexttestTestFixture 출력을 expected.txt로 저장하는 방법
2. 리팩토링 후 동일 출력 검증 방법
3. 커버리지 측정: mvn jacoco:report 연동

C++17

[T] C++ Google Test + 파일 비교로 Golden Master 구현:
1. 기존 출력을 expected_output.txt로 캡처
2. 리팩토링 후 출력과 파일 비교 테스트 작성
3. CMakeLists.txt에 ctest 통합 방법

💡 .cursorrules에 "항상 Java 21 / C++17 문법을 사용하고 경계값(quality 0-50, sellIn 0-1) 테스트를 포함해줘" 를 추가하면 모든 프롬프트에 자동 적용됩니다.

1단계

요구사항 분석 & 이해

Requirements Analysis — 비즈니스 규칙 파악 & 테스트 범위 정의

1

1단계 — 요구사항 분석 & Cursor AI 프롬프트

1단계

 **목적: 비즈니스 요구사항을 정확히 파악하고 테스트 범위를 정의**

Java — 요구사항 분석 프롬프트

Cursor Chat - @파일 컨텍스트 활용

@GildedRoseRequirements.txt @README.md

[P] 시니어 QA 엔지니어 관점에서
[C] Gilded Rose Java 프로젝트 (Java 21, JUnit 5)
[T] 요구사항 문서를 분석하여 정리해줘:
1. 각 아이템 타입별 비즈니스 규칙 (표 형식)
2. 예외 상황 및 경계값 조건
(quality: 0~50, sellIn 음수, Conjured)
3. Conjured 아이템 신규 요구사항
4. 테스트해야 할 주요 시나리오 목록
[F] Markdown 표 + 시나리오 번호 목록
결과를 requirements_analysis.md로 저장

C++17 — 요구사항 분석 프롬프트

Cursor Chat - C++17 동일 분석

@GildedRoseRequirements.txt @README.md

[P] 시니어 C++ QA 엔지니어
[C] Gilded Rose C++17, Google Test, CMake
[T] 요구사항 분석 후 C++ 구현 관점에서 정리:
1. 아이템 타입별 비즈니스 규칙
(enum class ItemType으로 표현 시)
2. C++17 std::string 비교 방식 주의사항
3. Conjured 아이템 추가 요구사항
4. Google Test 테스트 시나리오 목록
[F] Markdown + C++ 코드 스니펫 포함
requirements_analysis.md로 저장

1단계 핵심 체크포인트

☐ 비즈니스 규칙 정확성 5개 아이템 타입 규칙을 모두 표로 정리했는가?

☐ 테스트 범위 정의 Conjured 포함 신규 기능 요구사항까지 범위가 정의됐는가?

☐ 예외 상황 파악 quality 범위(0~50), sellIn 음수, Sulfuras 예외 모두 파악했는가?

☐ C++ 특이사항 확인 std::string 비교 시 == 연산자 vs compare() 차이 파악했는가?

 Chat에서 @GildedRoseRequirements.txt 를 직접 참조하면 AI가 실제 문서를 읽고 분석합니다. 분석 결과는 requirements_analysis.md로 저장하세요.

2단계

코드 품질 분석

(SOLID & Code Smell)

Code Quality Analysis — 현재 코드 문제점 파악

2

2단계 — 코드 품질 분석 프롬프트 (SOLID + Code Smell)

2단계

🔍 목적: SOLID 원칙 위반 및 코드 냄새 관점에서 현재 코드 문제점 파악

Java — SOLID 분석 프롬프트

Cursor Chat

@GildedRose.java

[P] 시니어 Java 아키텍트 + 클린코드 전문가
[T] updateQuality() 메서드를 분석해줘:
1. SRP 위반: 어떤 책임이 섞여 있는가?
2. OCP 위반: 새 아이템 추가 시 코드 수정 필요?
3. Code Smell 식별:
- Long Method (중첩 if 길이)
- Magic Number (0, 50, 11, 6)
- Duplicated Code (조건 반복)
4. 리팩토링 옵션으로 1~5 제안

C++ — SOLID 분석 프롬프트

Cursor Chat

@GildedRose.h @GildedRose.cpp

[P] 시니어 C++ 아키텍트
[T] updateQuality() 함수를 SOLID 관점 분석:
1. SRP 위반: if/else 체인의 책임 혼재
2. OCP 위반: 신규 아이템 추가 시 함수 수정
3. Code Smell:
- Magic Number (0, 50, 11, 6)
- 중첩 조건문 복잡도 (cyclomatic)
- C 스타일 문자열 비교 잔존 여부
4. C++17 modern style 리팩토링 방향 제안

🔍 GildedRose updateQuality() 주요 코드 스멜 — 사전 파악 체크리스트

Long Method

updateQuality() 내 중첩 if 5단계 이상
→ 인지 복잡도 MAX

Magic Number

0, 50, 11, 6 하드코딩 → 상수로 추출
필요

SRP 위반

1개 메서드가 5가지 아이템 로직 모두
처리

OCP 위반

Conjured 추가 시 if 블록 전체를 수정
해야 함

Duplicated Code

quality 범위 검사 (0~50) 코드 반복 5회
이상

Feature Envy

Item 필드에 과도하게 접근
(items[i].quality, items[i].name)



Cursor Chat: "@GildedRose.java 코드 스멜 분석 후 각 문제점의 심각도(Critical/Major/Minor)를 표로 정리하고 code_quality_report.md로 저장해줘"

3단계 테스트 계획

3단계

3단계 🌀 테스트 계획 수립 — 체계적 테스트 전략 & 케이스 설계

🌀 Java - 테스트 계획 프롬프트

@GildedRose.java @GildedRoseRequirements.txt

[T] GildedRose 테스트 계획을 수립해줘:

1. 단위 테스트(Unit) 범위 & 우선순위
JUnit 5 + @ParameterizedTest 활용
2. 경계값 테스트(CORRECT 원칙):
quality: 0, 1, 49, 50 / sellIn: 0, -1
3. 예외 상황 케이스 목록
4. JaCoCo 커버리지 90%+ 달성 전략

[F] 테스트 계획서 Markdown 표

🌀 C++ - 테스트 계획 프롬프트

@GildedRose.h @GildedRose.cpp

[T] C++ Google Test 테스트 계획 수립:

1. TEST_F 단위 테스트 범위 & 우선순위
2. EXPECT_EQ 경계값 케이스 목록:
quality: 0, 1, 49, 50 / sellIn: 0, -1
3. 예외 상황 (invalid_argument 등)
4. gcov/lcov 커버리지 90%+ 전략
5. CMakeLists.txt ctest 통합 방법

4단계 테스트 케이스 작성

4단계

4단계 🌀 테스트 케이스 작성 — 아이템 타입별 구체적 케이스

☕ Java - JUnit 5 케이스 생성 프롬프트

@GildedRoseTest.java @GildedRose.java

[T] 아이템 타입별 JUnit 5 테스트 케이스 작성:

1. 일반: quality -1, sellIn 0 후 -2, quality=0 유지
2. Aged Brie: quality +1, 기한 후 +2, 최대 50
3. Backstage Pass: sellIn 11→+2, 6→+3, 0→quality=0
4. Sulfuras: quality=80 불변, sellIn 불변
5. Conjured: quality -2, 기한 후 -4, 최소 0

각 케이스: @DisplayName + Given-When-Then
경계값 @CsvSource로 @ParameterizedTest 활용

⚙️ C++ - Google Test 케이스 생성 프롬프트

@GildedRoseTest.cpp @GildedRose.h

[T] Google Test 케이스를 구조적으로 작성:

```
struct GildedRoseTest : ::testing::Test {  
    // SetUp: 각 아이템 초기화  
};  
1. TEST_F(GildedRoseTest, NormalItem_QualityDecreases)  
2. TEST_F(GildedRoseTest, AgedBrie_QualityIncreases)  
3. TEST_F(GildedRoseTest, Backstage_Thresholds)  
4. TEST_F(GildedRoseTest, Sulfuras_NeverChanges)  
5. TEST_F(GildedRoseTest, Conjured_DoubleDegrade)  
EXPECT_EQ / ASSERT_EQ 경계값 포함  
ctest --output-on-failure 실행 검증
```



Cursor Composer(Ctrl+I): "위 5가지 아이템 타입의 테스트를 모두 작성하고 mvn test(Java) / ctest(C++) 실행해서 Green 확인해줘"

5단계 — 테스트 실행 & 결함 발견 (Defect Detection)

5단계

 목적: 실제 테스트 실행을 통한 결함 발견 · 원인 분석 · 심각도 평가

Java — 결함 분석 프롬프트

Cursor Chat - 테스트 실패 분석

@GildedRoseTest.java @GildedRose.java

[T] 다음 테스트 실패를 분석해줘:

GildedRoseTest.java의 foo() 테스트 실패 원인:

1. 실패 원인 파악 (실제 vs 예상 값 비교)
2. updateQuality()에서 해당 버그 위치 특정
3. 결함 심각도 평가 (Critical/Major/Minor)
4. 최소 코드 변경으로 수정 방안 제안
(Item 클래스는 수정 불가!)

수정 후 "mvn test" 실행해서 Green 확인해줘.

C++ — 결함 분석 프롬프트

Cursor Chat - C++ 결함 분석

@GildedRoseTest.cpp @GildedRose.cpp

[T] 실패하는 테스트를 분석해줘:

1. EXPECT_EQ 실패: 예상값 vs 실제값 비교
2. updateQuality() 내 버그 위치 특정
(Item 구조체 직접 수정은 금지!)
3. 결함 심각도 평가
4. C++17 스타일로 최소 변경 수정 방안

수정 후 "cmake --build build && ctest" 실행
에러 있으면 자동으로 수정해줘.

5단계 — 테스트 실행 & 결함 발견 (Defect Detection)

결함 심각도 분류 기준 (Defect Severity Matrix)

심각도	Java 예시	C++ 예시	설명
Critical	Sulfuras quality 변경됨	메모리 접근 위반	즉시 수정 필요. 핵심 비즈니스 로직 파괴
Major	Backstage Pass 구간 경계 오류	quality 음수 허용	기능 오동작. 다음 Sprint 내 수정
Minor	sellIn 0일 처리 순서 오류	매직 넘버 하드코딩	부분 오동작. 리팩토링 시 함께 수정
Info	불필요한 import 존재	네이밍 컨벤션 불일치	품질 개선 권고. 다음 리팩토링 시 반영

 테스트 실패 시 Cursor Chat에 @terminal 컨텍스트로 에러 로그를 직접 붙여넣으면 AI가 스택트레이스를 읽고 원인을 바로 분석합니다.

6단계 Golden Master 자동화

6단계

6단계 🌀 Golden Master — 회귀 테스트 강화 (TexttestFixture 기반)

🍷 Java - Golden Master 프롬프트

@TexttestFixture.java @GildedRose.java

[T] TexttestFixture를 활용한 Golden Master 구현:

1. TexttestFixture.java 실행 출력을 golden_master_expected.txt로 저장하는 JUnit 5 @BeforeAll 메서드 작성
2. 리팩토링 후 동일 출력 검증 테스트 작성
3. ApprovalsTest 프레임워크 활용 방안
4. mvn test로 자동 실행 + CI 통합 방법

[F] 완전한 Java 코드 + pom.xml 의존성

⚙️ C++ - Golden Master 프롬프트

@TexttestFixture.cpp @GildedRose.h

[T] C++ Google Test Golden Master 구현:

1. TexttestFixture 출력을 golden_master_expected.txt에 캡처하는 ::testing::Test SetUp 메서드 작성
2. 리팩토링 후 파일 비교 테스트: EXPECT_EQ(expected_output, actual_output)
3. CMakeLists.txt에 golden master 타겟 추가
4. GitHub Actions CI 통합 방법

[F] 완전한 C++ 코드 + CMakeLists.txt 수정

7단계 리팩토링 지원 테스트

7단계

7단계 리팩토링 지원 테스트 — 기능 회귀 방지 전략

Java - 리팩토링 검증 프롬프트

@GildedRose.java @GildedRoseTest.java

[T] Replace Type-code with Subclasses 리팩토링

전후 기능 동일성 검증 전략:

1. 리팩토링 전 Golden Master 출력 저장
2. ItemUpdater 인터페이스 + 서브클래스 분리 후 동일 출력 검증 테스트 작성
3. JaCoCo 커버리지 80%+ 유지 확인
4. 각 리팩토링 단계마다 "mvn test" Green 검증

[F] 단계별 체크리스트 + 테스트 코드

C++ - 리팩토링 검증 프롬프트

@GildedRose.h @GildedRose.cpp @GildedRoseTest.cpp

[T] 추상 클래스 기반 리팩토링 검증:

1. Golden Master 저장 → 추상화 리팩토링 → 동일 출력 검증 자동화
2. IGildedItem 인터페이스 분리 후 기존 모든 TEST_F 통과 확인
3. gcov 커버리지 변화 추적
4. 각 단계 커밋 후 ctest Green 보장

[F] C++17 코드 + CMakeLists.txt

 리팩토링 황금률: 테스트가 Green인 상태에서만 다음 리팩토링 단계로 진행하세요. Cursor Composer로 리팩토링 후 즉시 테스트 실행을 요청하세요.

8단계 🌀 결함 관리 & 보고 — 체계적 결함 추적 및 품질 메트릭

☕ Java - 결함 보고서 자동화 프롬프트

@GildedRose.java @GildedRoseTest.java

[T] GildedRose 프로젝트 결함 관리 문서 작성:

1. 결함 분류 체계 설계:
심각도(Critical/Major/Minor/Info)
x 아이템 타입(5종) 매트릭스 표
2. 결함 보고서 템플릿 (Markdown):
[결함ID] [심각도] [재현 방법] [기대/실제]
3. 품질 메트릭 수집 계획:
 - JUnit 5 실패율 추이
 - JaCoCo 커버리지 %
 - 결함 발견율 (단계별)

[F] defect_report.md로 저장

⚙️ C++ - 결함 보고서 프롬프트

@GildedRoseTest.cpp @GildedRose.cpp

[T] C++ 프로젝트 결함 관리 문서:

1. Google Test 실패 케이스 분류 표
2. 결함 템플릿:
[ID] [Severity] [Steps to Reproduce]
[Expected] [Actual] [Root Cause]
3. 품질 메트릭:
 - ctest 통과율
 - gcov 라인/브랜치 커버리지
 - 결함 밀도 (defects/KLOC)
4. GitHub Issues 연동 방법

[F] defect_report.md로 저장

9단계 QA 종합 검토 프롬프트

9단계

9단계 QA 종합 검토 — 전체 활동 효과성 검토 & Best Practice 도출

Cursor Chat - QA 종합 검토 프롬프트 (Java & C++ 공통)

@GildedRose.java @GildedRoseTest.java (또는 @GildedRose.cpp @GildedRoseTest.cpp)
@requirements_analysis.md @code_quality_report.md @defect_report.md

[P] QA 리드 엔지니어 관점에서

[T] Gilded Rose 프로젝트 전체 QA 활동을 종합 검토해줘:

1. 테스트 완료율 & 커버리지 분석 (목표 대비 달성률)
2. 발견된 결함의 패턴 분석 (아이템 타입별, 심각도별)
3. 9단계 QA 프로세스 중 가장 효과적이었던 단계 & 개선 필요 단계
4. 다음 레거시 프로젝트를 위한 Best Practice 5가지 도출
5. Cursor AI 활용 효과 측정 (시간 단축, 커버리지 향상, 결함 조기 발견)

[F] Markdown 보고서 (qa_final_report.md 저장)

각 섹션: 표 + 수치 + 구체적 권고사항 포함

💡 최종 보고서: 9단계 QA 활동 완료 후 qa_final_report.md를 prompting 브랜치에 커밋 → PR 생성 → 리뷰어 등록. 보고서가 곧 PR 설명입니다.

9단계 QA 체크포인트 종합표 & Cursor AI 핵심 명령어

종합 가이드

QA 활동별 핵심 체크포인트 & 산출물

단계	주요 활동	핵심 체크포인트	산출물
1단계	요구사항 분석	아이템 5종 규칙·경계값·Conjured 요구사항	requirements_analysis.md
2단계	코드 품질 분석	SOLID 위반·Code Smell 6종·리팩토링 우선순위	code_quality_report.md
3단계	테스트 계획	단위/통합/경계값 전략·커버리지 90%+ 목표	test_plan.md
4단계	테스트 케이스	5개 아이템 타입별 TC·@ParameterizedTest	GildedRoseTest.java/cpp
5단계	테스트 실행	결함 발견·재현·심각도 평가·수정 방안	defect_list.md
6단계	Golden Master	TexttestFixture 기반 회귀 자동화·CI 통합	GoldenMasterTest.java/cpp
7단계	리팩토링 지원	전후 기능 동일성·커버리지 유지·단계별 검증	refactoring_checklist.md
8단계	결함 관리	심각도 매트릭스·메트릭 수집·GitHub Issues	defect_report.md
9단계	QA 종합 검토	완료율·결함 패턴·Best Practice 5가지 도출	qa_final_report.md

Cursor AI 핵심 명령어

Ctrl+L	Chat 열기 (@파일 참조)
Ctrl+I	Composer 멀티파일 편집
Cmd+K	인라인 코드 수정 지시
@파일명	파일 컨텍스트 참조
@Codebase	전체 프로젝트 분석
/test	테스트 케이스 생성
/fix	버그 자동 수정
/explain	코드 상세 설명
Tab	자동완성 수락

.cursorrules 완전 설정 & Conjured 아이템 구현 비교

고급 설정

📁 .cursorrules (프로젝트 루트)

.cursorrules - Gilded Rose 전용 설정

```
# === 기술 스택 ===
# Java: Java 21 + JUnit 5 + Maven + JaCoCo
# C++: C++17 + Google Test + CMake + gcov

# === 절대 규칙 ===
- Item 클래스/구조체는 절대 수정 금지
- quality 범위: 0 이상 50 이하 (Sulfuras 제외)
- sellIn 음수 허용 (기한 초과 상태)

# === 테스트 규칙 ===
Java:
- 메서드명: should_[결과]_when_[조건]
- @DisplayName + Given-When-Then 구조
- 경계값 @CsvSource 필수 포함
C++:
- TEST_F(GildedRoseTest, [아이템]_[시나리오])
- EXPECT_EQ 경계값(0, 50, sellIn=0, -1)
- 각 테스트 독립적으로 실행 가능하게

# === 리팩토링 규칙 ===
- 리팩토링 전 반드시 테스트 Green 확인
- 매직 넘버 → 상수 (MAX_QUALITY=50 등)
- 결과는 Markdown 보고서로 저장
```

🌟 Conjured 아이템 구현 비교 (신규 기능)

☕ Java - Conjured 구현 프롬프트 결과

```
// Cursor AI 생성 결과 예시
private void updateConjuredItem(Item item) {
    int degradeRate = item.sellIn > 0 ? 2 : 4;
    item.quality = Math.max(0,
        item.quality - degradeRate);
}
// updateQuality()에서 호출:
// else if (item.name.contains("Conjured"))
//     updateConjuredItem(item);
```

⚙️ C++ - Conjured 구현 프롬프트 결과

```
// C++17 구현 예시
void updateConjured(Item& item) {
    const int rate = item.sellIn > 0 ? 2 : 4;
    item.quality = std::max(0,
        item.quality - rate);
}
// 호출: if (item.name.find("Conjured")
//         != std::string::npos)
//         updateConjured(item);
```

.cursorrules 완전 설정 & Conjured 아이템 구현 비교

고급 설정



Conjured 기능 추가 Composer 프롬프트

Ctrl+I - Composer 통합 구현

```
@GildedRose.java (또는 @GildedRose.cpp)  
@GildedRoseTest.java (또는 @GildedRoseTest.cpp)  
@GildedRoseRequirements.txt
```

Conjured 아이템 기능을 추가해줘:

- 규칙: 매일 quality -2, 기한 후 -4, 최소 0
- Item 클래스 수정 없이 구현
- 구현 후 전체 테스트 실행해서 Green 확인



Conjured 구현 핵심: Item 클래스를 수정할 수 없으므로 이름 문자열 기반 분기 처리. Cursor AI에 "Item 클래스 수정 없이" 조건을 명시하세요.

레거시 코드도 Cursor AI와 함께라면 두렵지 않다.

- ✅ 9단계 QA 프로세스: 요구사항 분석 → 코드 품질 → 테스트 → 결함 → Golden Master → 리팩토링 → 보고
- ✅ .cursorrules에 Item 수정 금지·quality 범위·경계값 규칙을 명시하면 모든 프롬프트 품질이 올라간다
- ✅ Java: @ParameterizedTest + JaCoCo / C++: Google Test + gcov — 언어·버전·도구를 항상 명시하라
- ✅ Conjured 구현이 진짜 시험: OCP를 지키면 기존 테스트 수정 없이 신규 기능이 추가된다

수 고 하 셧 습 니 다 🎉